

LoopCam

Full Development Plan for a Rolling-Buffer Dashcam Mobile App

Play / Stop / Save · Continuous background recording · Configurable buffer duration · Android (CameraX) + iOS (AVFoundation) + React Native shell

What this document covers

- The exact recording/buffer logic, generalized so the loop length is configurable (not fixed to 2 minutes).
- The three-button behavior (Play, Stop, Save) mapped to concrete state transitions.
- The full technical architecture for Android, iOS, and the React Native shell.
- How to keep the camera recording reliably in the background, and how to do it without draining the battery.
- Storage, file naming, and cleanup strategy.
- A phased build roadmap you can actually follow, from prototype to store release.

1. Core Concept

The app behaves like a dashcam: point the phone at the road, press Play once, and forget about it. The app is always holding only the most recent window of footage in memory/disk — never the whole trip. Nothing is written permanently until the user (or an automatic trigger) says **Save**.

1.1 The three actions

Action	What happens
Play	Starts continuous recording. The app begins filling a rolling buffer made of short clips. Once the buffer reaches its configured length, the oldest clip is deleted every time a new one finishes.
Stop	Recording stops immediately. The entire temporary buffer (all clips) is deleted. Nothing is kept.
Save	The current buffer (last N minutes) is merged into one file and written to permanent storage. Recording does not stop — a new buffer starts immediately, so the camera never goes dark.

This is exactly how dashcams and Tesla Sentry-style “save the last N minutes” features work — they never keep the full trip on disk, only a sliding window, and they commit that window to permanent storage only on request or on an automatic trigger (impact, button press).

2. Recording & Buffer Logic (Generalized Duration)

Your original spec used fixed numbers (2-minute buffer, 10-second clips). In production you want these to be configuration values, not hardcoded constants — so the same engine can offer a 30-second buffer for quick clips or a 10-minute buffer for long drives, without touching the recording code.

2.1 Two independent settings

Setting	Meaning	Typical range	Effect of changing it
clip_duration_sec	Length of each individual recorded segment	5 – 30 sec	Smaller = finer-grained deletion, more files, slightly more overhead. Larger = fewer files, coarser save boundaries.
buffer_duration_sec	Total length of footage kept before Save	30 sec – 15 min	Directly drives device_count = $\text{buffer_duration_sec} / \text{clip_duration_sec}$

Everything downstream — how many clips are kept, when the oldest is deleted, how big the save file is — is derived from these two numbers. This is the key change that makes the logic scale from a 30-second buffer to a 10-minute buffer with no code change.

2.2 Core formula

```
max_clips = ceil(buffer_duration_sec / clip_duration_sec)

# Example presets
Quick clip  -> clip_duration_sec = 5,  buffer_duration_sec = 30  -> max_clips = 6
Default    -> clip_duration_sec = 10, buffer_duration_sec = 120 -> max_clips = 12
Long trip   -> clip_duration_sec = 15, buffer_duration_sec = 600 -> max_clips = 40
```

2.3 State machine

```
STATES: IDLE -> RECORDING -> (SAVING | STOPPING) -> back to IDLE or RECORDING

IDLE
  on PLAY -> clear any leftover temp clips, start Segment Loop, go to RECORDING

RECORDING
  on each clip finished:
    buffer.append(new_clip)
    if buffer.length > max_clips:
      delete_from_disk(buffer.pop_oldest())
    start_next_clip()          # loop continues automatically

  on STOP -> cancel current clip recording
    delete_from_disk(all clips in buffer)
    buffer.clear()
    go to IDLE

  on SAVE -> snapshot = copy(buffer)          # freeze current window
```

```
spawn_async: merge(snapshot) -> /Videos/Incident_<timestamp>.mp4
buffer.clear()                # start a fresh window immediately
stay in RECORDING (no interruption to the live feed)
```

on IMPACT DETECTED (optional, see section 8) -> behave exactly like SAVE

The important design decision here: **Save never pauses recording.** The merge-and-write step runs on a background thread/queue while a brand-new buffer starts filling immediately, so the live camera feed is continuous and the user never sees a gap — even if they hit Save five times in a row.

2.4 Buffer as a ring, not a list you keep re-scanning

Implement the buffer as a fixed-capacity circular queue (deque with maxlen = max_clips). Appending a clip when the queue is full automatically evicts the oldest one — you get the delete-oldest behavior for free from the data structure instead of writing manual cleanup logic that can drift out of sync with disk state.

```
buffer = Deque(maxLen: max_clips)

function onClipFinished(clip):
  if buffer.isFull():
    evicted = buffer.peekOldest()
    fileSystem.delete(evicted.path)
  buffer.push(clip)
```

3. System Architecture

Recording reliability is the hard part of this app, so the camera engine must live in native code (Kotlin on Android, Swift on iOS), not in JavaScript. React Native is only the control surface: buttons, settings, gallery of saved clips, and a thin bridge that calls into the native recorder.

```
React Native (JS/TS)
|-- UI: Play / Stop / Save buttons, live preview, settings, saved-clips gallery
|-- Native Module Bridge (TurboModule / JSI)
    |
    |-----|
    v               v
ANDROID NATIVE (Kotlin)      iOS NATIVE (Swift)
CameraX -> VideoCapture      AVCaptureSession -> AVAssetWriter
Segment Controller (ring buffer)
Foreground Service (background survival)
Storage Manager (app-private dir)
WorkManager (cleanup / battery-safe tasks)

Background Audio/Camera mode
Storage Manager (app sandbox dir)
BGTaskScheduler (best-effort cleanup)

Shared native layer
-> Merge engine (concat clips into one file on Save)
-> Metadata writer (GPS, timestamp, speed as JSON sidecar)
-> Event emitter back to JS (buffer state, storage warnings, errors)
```

3.1 Why not do the recording logic in JavaScript

- React Native's JS thread can be paused by the OS or blocked by UI work — a dropped frame in the buffer loop means lost footage or a corrupted clip boundary.
- Background execution guarantees (foreground services on Android, background modes on iOS) are OS-level contracts that only native code can properly declare and hold.
- MediaRecorder/AVAssetWriter need tight, low-latency control over segment start/stop timing that a JS bridge round-trip isn't built for.

4. Technology Stack

Layer	Android	iOS
Camera capture	CameraX (VideoCapture use case)	AVCaptureSession
Encoding / writing	MediaRecorder or MediaCodec + MediaMuxer	AVAssetWriter
Segment merging	MediaMuxer (or FFmpeg-kit if re-encoding is needed)	AVAssetExportSession (or FFmpeg-kit)
Background survival	Foreground Service + persistent notification	Background Modes: Audio, Camera (limited)
Storage	App-private external storage (scoped storage)	App sandbox Documents/ directory
Deferred/cleanup work	WorkManager	BGTaskScheduler / BGProcessingTask
Sensors (impact detect)	SensorManager (accelerometer, gyroscope)	Core Motion
Location	FusedLocationProviderClient	CoreLocation
Cross-platform shell	React Native (New Architecture / TurboModules)	same

Camera libraries such as react-native-vision-camera are useful for the live preview and simple capture, but for a rolling buffer with guaranteed background survival, plan on writing a custom native module for the recording engine rather than relying on a JS camera library to manage the loop.

5. Running Reliably in the Background

This is the single hardest engineering problem in the app. "Just put the phone down and it keeps recording" requires the OS to trust the app enough not to kill it, and both Android and iOS actively try to kill background camera/CPU work to save battery.

5.1 Android — Foreground Service

While recording, run a Foreground Service of type camera (and location if GPS tagging is on). This shows a persistent notification ("LoopCam is recording") and tells Android's process manager this task must not be killed opportunistically.

- Declare `foregroundServiceType="camera|location"` in the manifest (required on Android 14+).
- Request the runtime permission `FOREGROUND_SERVICE_CAMERA`.
- Ask the user to exempt the app from battery optimization ("Unrestricted" battery mode) on first launch — without this, aggressive OEM battery managers (Xiaomi/Huawei/Samsung) will still kill the service.
- Keep the notification lightweight: elapsed time + Stop/Save quick actions, updated at most once per second.

5.2 iOS — Background Modes (and the real limits)

iOS is intentionally stricter. There is no general "keep recording indefinitely from the lock screen" guarantee the way Android's foreground service gives you.

- Enable Background Modes: **Audio, AirPlay, and Picture in Picture** — an active audio session is the standard trick that keeps an app alive in the background, since iOS treats ongoing audio as a legitimate reason to stay resident.
- Camera capture itself is only reliably guaranteed in the **foreground** or with the screen on. For true background/lock-screen recording, most production dashcam apps on iOS keep the screen on (a dedicated "driving screen") rather than promising lock-screen recording.
- Be explicit with users in-app: on iOS, best practice is "keep the app open and the screen on while driving," mounted on a dashboard holder — set this expectation during onboarding so it isn't a surprise.
- Apple App Review will scrutinize any background camera/location claim — be ready to justify it as core functionality (dashcam) in the review notes, and avoid requesting more background capability than the feature needs.

5.3 Practical recommendation

Android: true background/lock-screen recording is achievable and is the flagship experience. iOS: ship a "screen-on driving mode" (auto-locks disabled, dimmed display) as the primary mode, and treat any further background capability as a bonus, not a promise.

6. Battery Efficiency Strategy

Technique	Why it helps
Hardware encoder only (no software re-encode during recording)	MediaCodec/AVAssetWriter use the device's dedicated video encoder chip — orders of magnitude more power-efficient than CPU-based encoding.
Lower resolution/bitrate default (1080p @ ~5 Mbps, or 720p for battery-saver mode)	Cuts encoder + storage I/O load; user can opt into 4K if plugged into power.
Turn the screen off / dim it during recording (Android)	Display is one of the largest power draws; Android can keep the foreground service running with the screen off.
Batch/deferred cleanup with WorkManager instead of a live timer thread	Avoids waking the CPU every second just to check the buffer; deletion is tied to the natural clip-finished event instead.
Auto-detect car charger / plugged-in power	Offer a "Continuous mode" (longer buffer, higher resolution) only when charging is detected; fall back to a conservative preset on battery.
Auto-stop on low battery threshold (e.g. 15%)	Prevents the phone from fully dying mid-drive; auto-Save the current buffer first so nothing is lost.
GPS at reduced sampling rate (1 fix / 2–3 sec) instead of max frequency	Location metadata doesn't need survey-grade precision; this alone meaningfully cuts GPS power draw.
Merge (Save) operations queued, not instant	Video concatenation is CPU work; running it through a background task queue avoids competing with the live recording thread.

7. Storage Management

7.1 File naming & layout

```

App private storage (temp buffer, auto-cleaned):
  /LoopCam/tmp/session_<sessionId>/clip_0001.mp4
                                     clip_0002.mp4 ...

Permanent saved storage (after SAVE):
  /LoopCam/saved/Incident_2026-08-03_16-20-11.mp4
  /LoopCam/saved/Incident_2026-08-03_16-20-11.json    <- GPS/speed/timestamp metadata

On STOP:
  delete entire /LoopCam/tmp/session_<sessionId>/ directory

```

7.2 Storage budget (example at default preset)

Quality	Bitrate	Per-minute size	2-min buffer	10-min buffer
720p	~2.5 Mbps	~19 MB	~38 MB	~190 MB
1080p	~5 Mbps	~37 MB	~75 MB	~375 MB

Quality	Bitrate	Per-minute size	2-min buffer	10-min buffer
4K (plugged-in only)	~20 Mbps	~150 MB	~300 MB	~1.5 GB

Guard rails to add: warn the user below ~1 GB free space, cap total saved clips (e.g. auto-delete oldest saved clip past a configurable max like 5 GB or 50 clips, unless the user marks it "protected"), and never let the temp buffer exceed its configured max_clips regardless of app restarts (recover/clean on launch).

8. Optional Features Worth Planning For

Feature	What it adds	Effort
Automatic impact detection	Accelerometer/gyroscope spike triggers an automatic SAVE, like Tesla Sentry / most dashcams	Medium
GPS + speed overlay	Sidecar JSON per saved clip: lat/lng/speed/timestamp, optional on-video watermark	Low–Medium
Parked mode / motion-triggered recording	Lower frame rate standby that only ramps up on detected motion near the car	High
Cloud backup of saved clips	Auto-upload Saved (not buffer) footage to cloud storage over Wi-Fi	Medium
Multi-camera (front + interior)	Second capture session in parallel; roughly doubles encoder/battery/storage cost	High

Recommendation: build impact-detection and GPS metadata into v1 since they reuse sensors you'll already want for a "real" dashcam app; treat parked mode, cloud backup, and multi-camera as v2 features.

9. Development Roadmap

Phase	Goal	Key deliverables
0 — Spike (1–2 wks)	Prove the loop works at all	Native Android prototype: fixed clip size, ring buffer, manual Play/Stop/Save via debug buttons. No RN yet.
1 — Core engine (2–3 wks)	Configurable buffer, both platforms	clip_duration/buffer_duration config, ring-buffer eviction, Save merges to permanent file, Stop wipes temp. Android + iOS native modules.
2 — Background survival (2 wks)	Recording keeps going when app is backgrounded	Foreground Service (Android) with persistent notification; iOS "driving mode" screen-on flow; battery-optimization exemption prompts.
3 — RN shell + UX (2 wks)	Usable app	Play/Stop/Save UI, live preview, saved-clips gallery, settings (resolution, buffer length, storage limit).

Phase	Goal	Key deliverables
4 — Storage & battery hardening (1–2 wks)	Won't fill disk or kill battery	Storage budget enforcement, low-battery auto-save-and-stop, resolution presets, orphaned-temp-file recovery on launch.
5 — Sensors (1–2 wks)	Automatic Save on impact + GPS tagging	Accelerometer threshold trigger, JSON metadata sidecar, optional on-video watermark.
6 — Polish & store prep (2 wks)	Ship-ready	Permission rationale screens, App Review notes for background camera/location, crash reporting, beta test on real drives.

10. Key Risks & Mitigations

Risk	Mitigation
OEM battery managers (Xiaomi/Huawei/Samsung) kill the service anyway	Detect manufacturer, deep-link user to the specific battery-whitelist settings screen on first run.
App Review rejection over background camera/location use	Frame as core dashcam functionality in review notes; avoid requesting always-on background beyond what recording needs; provide a demo video/account.
Corrupted merge if Save happens mid-clip-write	Only merge clips that are fully closed/flushed; the in-progress clip stays out of the snapshot and joins the next buffer.
Storage fills up on long trips	Enforce storage budget from section 7.2; auto-warn and auto-stop before disk is exhausted.
User expectation mismatch on iOS (thinks it records like Android in background)	Clear onboarding messaging: iOS = keep screen on while driving.

This plan gives you a buildable path: start with Phase 0 as a throwaway native prototype to validate the ring-buffer timing on a real device before investing in the React Native shell or iOS background handling.